

Level-1 Problem 1: Student Score Analyzer Using Arrays and Maps

Scenario:

A training institute wants to analyze student scores stored in an array. The system should calculate total marks, average, highest score, and count of students scoring above a threshold.

Requirements:

- Store student marks in an array.
- Use array methods (push, map, filter, reduce) for processing.
- Store subject-wise highest marks using a Map (key-value pair).
- Display total, average, and filtered results.

Technical Constraints:

- Must use array indexing and iteration.
- Use reduce() for total calculation.
- Use filter() for threshold-based filtering.
- Use Map or Dictionary for subject-highest mapping.

Sample Input:

Marks: [78, 85, 90, 67, 88]

Threshold: 80

Sample Output:

Total Marks: 408

Average Marks: 81.6

Students above 80: 3

Highest Score: 90

Expectations:

- Clean and modular implementation.
- Proper use of array methods.

Code Screenshot:

Program.cs*  

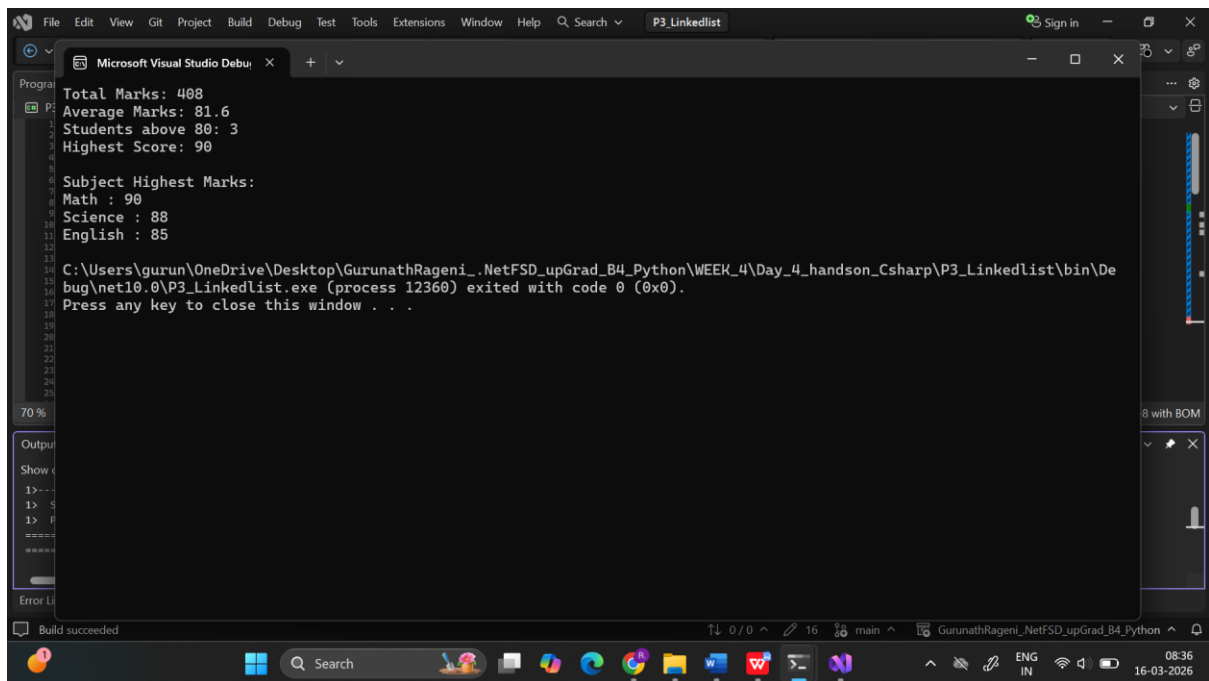
P1_Array_methods

 StudentScoreAnalyzer.P

```
31 using System;
32 namespace StudentScoreAnalyzer
33 {
34     1 reference
35     class Program
36     {
37         0 references
38         static void Main(string[] args)
39         {
40             // Marks array
41             int[] marks = { 78, 85, 90, 67, 88 };
42             int threshold = 80;
43             // Total marks (reduce equivalent)
44             int totalMarks = marks.Sum();
45             // Average marks
46             double averageMarks = marks.Average();
47
48             // Students above threshold (filter equivalent)
49             var aboveThreshold = marks.Where(m => m > threshold);
50
51             // Highest score
52             int highestScore = marks.Max();
53             // Dictionary for subject-wise highest marks
54             Dictionary<string, int> subjectHighest = new Dictionary<string, int>();
55             subjectHighest["Math"] = 90;
56             subjectHighest["Science"] = 88;
57             subjectHighest["English"] = 85;
58             // Output
59             Console.WriteLine("Total Marks: " + totalMarks);
60             Console.WriteLine("Average Marks: " + averageMarks);
61             Console.WriteLine("Students above 80: " + aboveThreshold.Count());
62             Console.WriteLine("Highest Score: " + highestScore);
63             Console.WriteLine("\nSubject Highest Marks:");
64
65             foreach (var item in subjectHighest)
66             {
67                 Console.WriteLine(item.Key + " : " + item.Value);
68             }
69         }
70     }
71 }
```

70 %  No issues found  Ready Search

Output Screenshot:



```
Microsoft Visual Studio Debu
P3_Linkedlist

Total Marks: 408
Average Marks: 81.6
Students above 80: 3
Highest Score: 90

Subject Highest Marks:
Math : 90
Science : 88
English : 85

C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_4_handson_Csharp\P3_Linkedlist\bin\De
bug\net10.0\P3_Linkedlist.exe (process 12360) exited with code 0 (0x0).
Press any key to close this window . . .

Build succeeded
0 / 0 16 main GurunathRageni_.NetFSD_upGrad_B4_Python 08:36 16-03-2026
```

Level-1 Problem 2: Stack-Based Undo System

Scenario:

Design a simple text editor undo feature using Stack (LIFO principle).

Requirements:

- Implement stack using arrays.
- Support push (add action) and pop (undo action).

- Display current state after each operation.

Technical Constraints:

- Only array-based stack implementation.
- Must follow LIFO order strictly.
- Handle empty stack condition.

Sample Input:

Actions: Type A, Type B, Type C, Undo, Undo

Sample Output:

Current State After Operations: Type A

Expectations:

- Correct LIFO implementation.
- Proper error handling.
- Clear logic structure.

Learning Outcome:

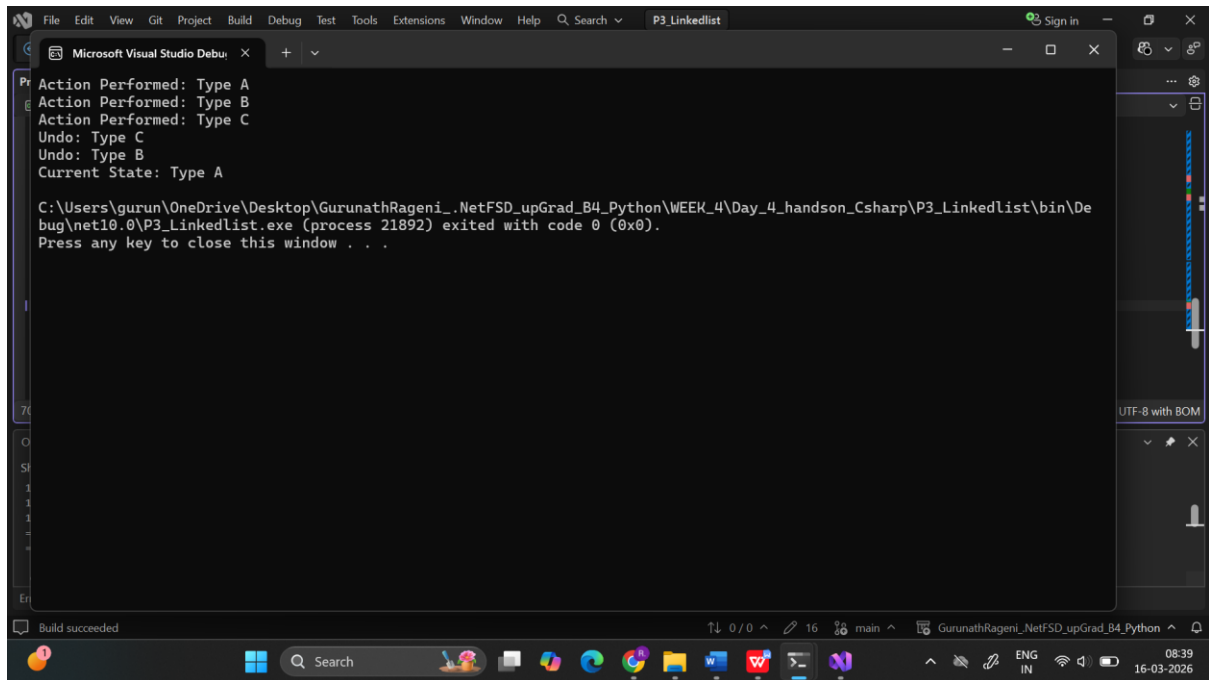
- Understand stack operations.
- Learn LIFO principle application.
- Implement stack using arrays.

Code screenshot:

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P2_Stack_based_undo Sign in
Debug Any CPU P2_Stack_based_undo P2_Stack_based_undo.Main(string[] args)
Program.cs
P2_Stack_based_undo
- Implement stack using arrays.*/
using System;
using System.Collections.Generic;
using System.Text;
namespace P2_Stack_based_undo
{
    internal class Class1
    {
        using System;
    }
    namespace StackUndoSystem
    {
        class Program
        {
            static void Main(string[] args)
            {
                string[] stack = new string[10]; // Array stack
                int top = -1; // Stack pointer
                // Push function
                void Push(string action)
                {
                    if (top == stack.Length - 1)
                    {
                        Console.WriteLine("Stack Overflow");
                        return;
                    }
                    top++;
                    stack[top] = action;
                    Console.WriteLine("Action Performed: " + action);
                }
                // Pop function (Undo)
                void Pop()
                {
                    if (top == -1)
                }
            }
        }
    }
}
Ln: 44, Ch: 44 SPC CRLF UTF-8 with BOM
Ready 0/0 14 main GurunathiRageni_NetFSD_upGrad_B4_Python 08:26 16-03-2026
```

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P2_Stack_based_undo Sign in
Debug Any CPU P2_Stack_based_undo P2_Stack_based_undo.Main(string[] args)
Program.cs
P2_Stack_based_undo
// Pop function (Undo)
void Pop()
{
    if (top == -1)
    {
        Console.WriteLine("Nothing to Undo (Stack Empty)");
        return;
    }
    Console.WriteLine("Undo: " + stack[top]);
    top--;
}
// Display current state
void Display()
{
    if (top == -1)
    {
        Console.WriteLine("Editor Empty");
        return;
    }
    Console.WriteLine("Current State: ");
    for (int i = 0; i <= top; i++)
    {
        Console.Write(stack[i] + " ");
    }
    Console.WriteLine();
}
// Sample operations
Push("Type A");
Push("Type B");
Push("Type C");
Pop(); // Undo
Pop(); // Undo
Display();
}
}
Ln: 102, Ch: 2 SPC CRLF UTF-8 with BOM
Ready 0/0 15 main GurunathiRageni_NetFSD_upGrad_B4_Python 08:26 16-03-2026
```

Output Screenshot:



```
Microsoft Visual Studio Debug Console
Pr Action Performed: Type A
E Action Performed: Type B
Action Performed: Type C
Undo: Type C
Undo: Type B
Current State: Type A

C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_4_handson_Csharp\P3_LinkedList\bin\De
bug\net10.0\P3_LinkedList.exe (process 21892) exited with code 0 (0x0).
Press any key to close this window . . .

70
St
1
1
1
1
+
Er

Build succeeded
0 / 0 ^ 16 main ^ GurunathRageni_.NetFSD_upGrad_B4_Python ^
Search
ENG IN 08:39 16-03-2026
```

Level-2 Problem 1: Employee Management Using Linked List

Scenario:

A company wants to maintain employee records dynamically using a Linked List structure.

Requirements:

- Create Node structure with employee ID and name.
- Implement insertion at beginning and end.
- Implement deletion by employee ID.
- Traverse and display employee list.

Technical Constraints:

- Must implement singly linked list.
- No use of built-in list structures.
- Proper memory handling and pointer updates.

Sample Input:

Insert: (101, John), (102, Sara), (103, Mike)

Delete: 102

Sample Output:

Employee List After Deletion:

101 - John

103 – Mike

Expectations:

- Correct node linking.
- Efficient traversal logic.
- Clean insertion and deletion operations.

Learning Outcome:

- Understand linked list structure.
- Perform insertion and deletion operations.
- Learn dynamic data structure behavior.

Code screenshot:

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P3_Linkedlist Sign in
Debug Any CPU P3_Linkedlist
Program.cs P3_Linkedlist Node empld
31 using System;
32
33 // reference
34 class Node
35 {
36     public int empld;
37     public string name;
38     public Node next;
39
40     // 2 references
41     public Node(int id, string name)
42     {
43         this.empld = id;
44         this.name = name;
45         this.next = null;
46     }
47
48 // 2 references
49 class EmployeeLinkedList
50 {
51     Node head;
52
53     // Insert at Beginning
54     // 0 references
55     public void InsertAtBeginning(int id, string name)
56     {
57         Node newNode = new Node(id, name);
58         newNode.next = head;
59         head = newNode;
60     }
61
62     // Insert at End
63     // 3 references
64     public void InsertAtEnd(int id, string name)
65     {
66         Node newNode = new Node(id, name);
67
68         if (head == null)
69         {
70             head = newNode;
71             return;
72         }
73     }
74
75     // 0/0 ^ 15 main ^ GurunathiRageni_NetFSD_upGrad_B4_Python ^
76 Ln: 29, Ch: 41 SPC CRLF UTF-8 with BOM
77 Ready 08:28 16-03-2026
```

```
File Edit View Git Project Build Debug Test Tools Extensions Window Help Search P3_Linkedlist Sign in
Debug Any CPU P3_Linkedlist
Program.cs P3_Linkedlist Node empld
65     head = newNode;
66     return;
67 }
68
69 Node temp = head;
70 while (temp.next != null)
71 {
72     temp = temp.next;
73 }
74 temp.next = newNode;
75 }
76
77 // Delete by Employee ID
78 // 1 reference
79 public void Delete(int id)
80 {
81     if (head == null)
82     {
83         return;
84     }
85
86     // If head node needs deletion
87     if (head.empld == id)
88     {
89         head = head.next;
90         return;
91     }
92
93     Node temp = head;
94     while (temp.next != null && temp.next.empld != id)
95     {
96         temp = temp.next;
97     }
98
99     if (temp.next != null)
100     {
101         temp.next = temp.next.next;
102     }
103 }
104
105 // Display Employees
106
107 // 0/0 ^ 15 main ^ GurunathiRageni_NetFSD_upGrad_B4_Python ^
108 Ln: 29, Ch: 41 SPC CRLF UTF-8 with BOM
109 Ready 08:28 16-03-2026
```



```
103 }
104
105 // Display Employees
106 // reference
107 public void Display()
108 {
109     Node temp = head;
110     while (temp != null)
111     {
112         Console.WriteLine(temp.empId + " - " + temp.name);
113         temp = temp.next;
114     }
115 }
116
117 class Program
118 {
119     static void Main(string[] args)
120     {
121         EmployeeLinkedList list = new EmployeeLinkedList();
122
123         // Insert Employees
124         list.InsertAtEnd(101, "John");
125         list.InsertAtEnd(102, "Sara");
126         list.InsertAtEnd(103, "Mike");
127
128         // Delete employee with ID 102
129         list.Delete(102);
130
131         Console.WriteLine("Employee List After Deletion:");
132
133         // Display List
134         list.Display();
135     }
136 }
```

Output Screenshot:

```
Employee List After Deletion:
101 - John
103 - Mike

C:\Users\gurun\OneDrive\Desktop\GurunathRageni_.NetFSD_upGrad_B4_Python\WEEK_4\Day_4_handson_Csharp\P3_Linkedlist\bin\De
bug\net10.0\P3_Linkedlist.exe (process 14516) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .|
```

Level-2 Problem 2: Student Record Management System Using Record Data Structure

Scenario:

A college wants to develop a console-based Student Record Management System. The system should store and manage student records using a Record data structure. Each student record should contain fields such as Roll Number, Name, Course, and Marks. The system must allow users to add multiple student records, display all records, and search for a student using the Roll Number.

Requirements:

1. Define a Record to store student details.
2. Allow the user to input details for multiple students.
3. Display all student records in a formatted manner.
4. Provide a search functionality to find a student by Roll Number.
5. Display appropriate message if the record is not found.

Technical Constraints:

1. Must use Record data type.
2. Use an array (or list) of records to store multiple students.
3. Do not use external databases.
4. Program should be menu-driven.
5. Input validation must be handled for Roll Number and Marks.

Sample Input:

Enter number of students: 2

Enter Roll Number: 101

Enter Name: Aisha

Enter Course: BCA

Enter Marks: 85

Enter Roll Number: 102

Enter Name: Rahul

Enter Course: BSc

Enter Marks: 78

Search Roll Number: 101

Sample Output:

Student Records:

Roll No: 101 | Name: Aisha | Course: BCA | Marks: 85

Roll No: 102 | Name: Rahul | Course: BSc | Marks: 78

Search Result:

Student Found:

Roll No: 101 | Name: Aisha | Course: BCA | Marks: 85

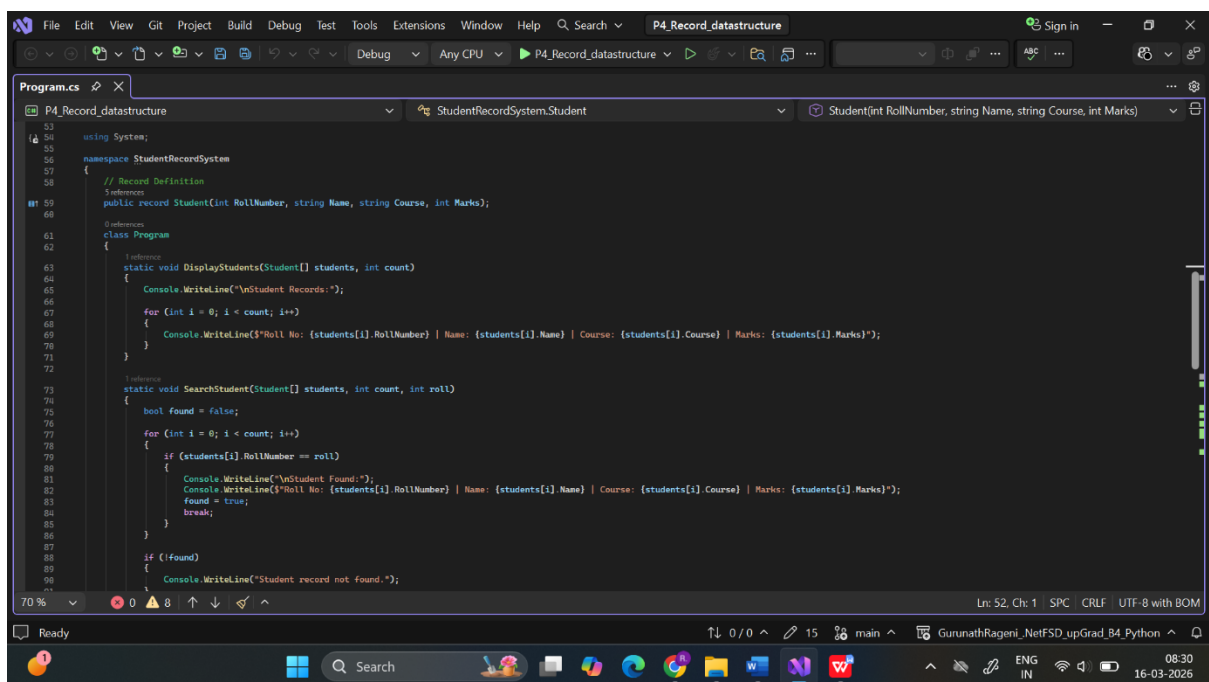
Expectations:

1. Proper use of Record/Structure syntax.
2. Clean and modular code.
3. Proper formatting of displayed output.
4. Efficient search implementation.

Learning Outcome:

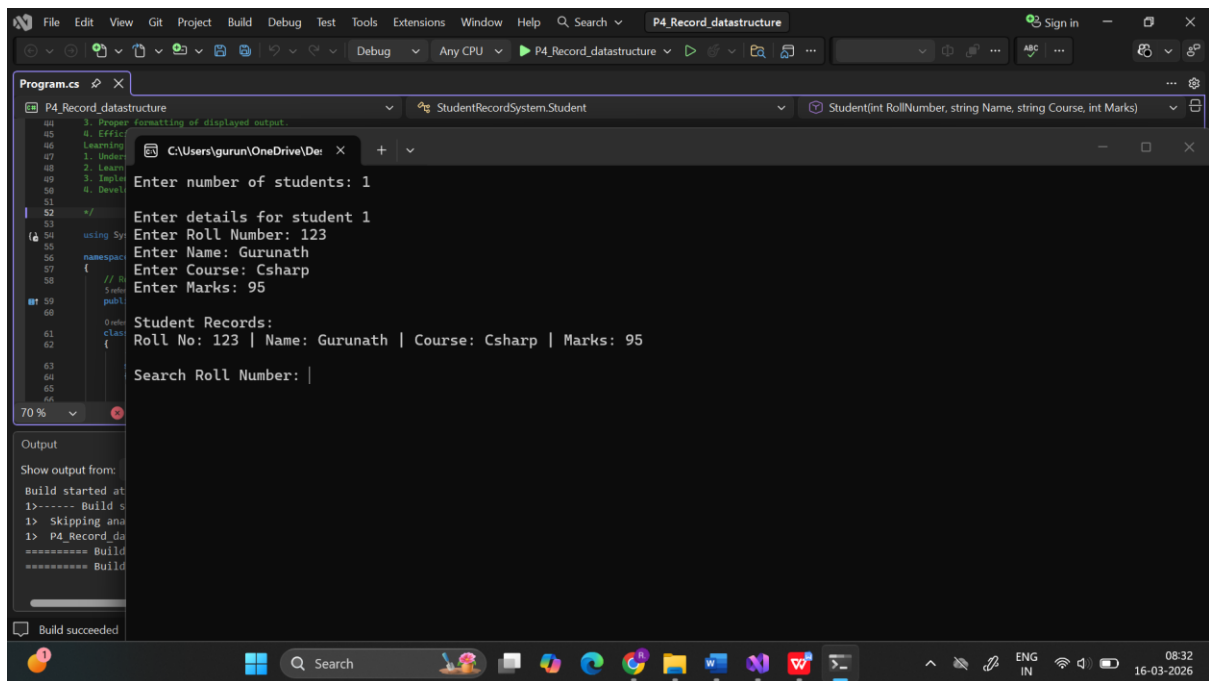
1. Understand how to define and use Record/Structure data types.
2. Learn how to manage multiple records using arrays/lists.
3. Implement searching techniques on structured data.
4. Develop structured problem-solving skills for real-world scenarios.

Code screenshot:



```
53 using System;
54
55 namespace StudentRecordSystem
56 {
57     // Record Definition
58     public record Student(int RollNumber, string Name, string Course, int Marks);
59
60     // Program
61     class Program
62     {
63         static void DisplayStudents(Student[] students, int count)
64         {
65             Console.WriteLine("\nStudent Records:");
66             for (int i = 0; i < count; i++)
67             {
68                 Console.WriteLine($"Roll No: {students[i].RollNumber} | Name: {students[i].Name} | Course: {students[i].Course} | Marks: {students[i].Marks}");
69             }
70         }
71
72         static void SearchStudent(Student[] students, int count, int roll)
73         {
74             bool found = false;
75             for (int i = 0; i < count; i++)
76             {
77                 if (students[i].RollNumber == roll)
78                 {
79                     Console.WriteLine("\nStudent Found:");
80                     Console.WriteLine($"Roll No: {students[i].RollNumber} | Name: {students[i].Name} | Course: {students[i].Course} | Marks: {students[i].Marks}");
81                     found = true;
82                     break;
83                 }
84             }
85             if (!found)
86             {
87                 Console.WriteLine("Student record not found.");
88             }
89         }
90     }
91 }
```


Code output:



The screenshot shows the Visual Studio IDE with a C# program running. The code in `Program.cs` defines a `Student` class and a `StudentRecordSystem` class. The `StudentRecordSystem` class has methods to add students, display records, and search for a student by roll number. The output window shows the program's execution, including prompts for the number of students, student details, and the resulting student records.

```
Program.cs
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace P4_Record_datastructure
8  {
9      class Student
10     {
11         public int RollNumber;
12         public string Name;
13         public string Course;
14         public int Marks;
15     }
16
17     class StudentRecordSystem
18     {
19         List<Student> students = new List<Student>();
20
21         void AddStudent()
22         {
23             int n;
24             Console.WriteLine("Enter number of students:");
25             n = Convert.ToInt32(Console.ReadLine());
26
27             for (int i = 0; i < n; i++)
28             {
29                 Console.WriteLine("Enter details for student {0}", i + 1);
30                 Student s = new Student();
31                 s.RollNumber = Convert.ToInt32(Console.ReadLine());
32                 s.Name = Console.ReadLine();
33                 s.Course = Console.ReadLine();
34                 s.Marks = Convert.ToInt32(Console.ReadLine());
35                 students.Add(s);
36             }
37
38             Console.WriteLine("Student Records:");
39             DisplayRecords();
40
41             Console.WriteLine("Search Roll Number:");
42             int searchRollNumber = Convert.ToInt32(Console.ReadLine());
43             SearchStudent(searchRollNumber);
44         }
45
46         void DisplayRecords()
47         {
48             foreach (Student s in students)
49             {
50                 Console.WriteLine("Roll No: {0} | Name: {1} | Course: {2} | Marks: {3}", s.RollNumber, s.Name, s.Course, s.Marks);
51             }
52         }
53
54         void SearchStudent(int searchRollNumber)
55         {
56             Student s = students.FirstOrDefault(stud => stud.RollNumber == searchRollNumber);
57             if (s != null)
58             {
59                 Console.WriteLine("Student found: {0} | {1} | {2} | {3}", s.RollNumber, s.Name, s.Course, s.Marks);
60             }
61             else
62             {
63                 Console.WriteLine("Student not found");
64             }
65         }
66     }
67 }
68
69 // Program execution output:
70 Enter number of students: 1
71 Enter details for student 1
72 Enter Roll Number: 123
73 Enter Name: Gurunath
74 Enter Course: Csharp
75 Enter Marks: 95
76
77 Student Records:
78 Roll No: 123 | Name: Gurunath | Course: Csharp | Marks: 95
79
80 Search Roll Number: |
```

Output

Show output from:

Build started at 16-03-2026 08:32:15

1>----- Build succeeded

1> Skipping analysis

1> P4_Record_datastructure

----- Build succeeded

Build succeeded